Contribution Guide and Engineering Standards

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-18
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	`com.aslam.mediareminder`
Purpose	Define repository workflow, code quality, architecture boundaries, tests, documentation, dependency review and definition of done.

> **Reading rule:** This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but MUST NOT be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Contribution values

Correctness, privacy, accessibility and battery behavior outrank code novelty. Contributions should reduce ambiguity, preserve portable backups and keep native alarm controls available without JavaScript. The project welcomes focused changes with evidence.

2 Repository setup

Expected tools are recorded and pinned in the repository:

- current supported Node LTS compatible with React Native 0.86.x;
- npm lockfile with `npm ci`;
- JDK version required by the selected Android Gradle Plugin;
- Android SDK platforms 26, 36 and 37 plus emulator images used in CI;
- Ruby/CocoaPods are not required for Android-only v1;
- Git and platform signing tools.

Canonical commands:

```
npm ci
npm run typecheck
npm run lint
npm test
npm run android
cd android && ./gradlew test lintDebug connectedCheck
```

Release commands are scripted and never request signing secrets from source-controlled files.

3 Branch and change workflow

Use short-lived branches from `main`. `main` stays buildable. Pull requests must be focused, linked to requirement/issue and updated from current main. Direct pushes to protected release tags are forbidden.

Commit messages follow Conventional Commits, for example:

```
feat(alarm): add native snooze idempotency
```

A commit history may be squashed, but final release notes preserve meaningful changes.

4 Architecture boundaries

Contributors MUST follow MR-07:

- no durable reminder state in React-only storage;
- no AlarmManager calls from JavaScript packages;
- no raw DAO access from Android activities/receivers;
- no media byte arrays crossing the bridge;
- no arbitrary filesystem path exposed to UI;
- no RN dependency in Play/Snooze/Dismiss receiver path;
- no notification/full-screen behavior implemented with overlay permission;
- no direct archive-to-database mapping without validator/repository.

CI includes dependency/architecture tests where practical.

5 TypeScript standards

- `strict` enabled; avoid `any` and unsafe casts.
- Runtime-decode native/external payloads where Codegen cannot guarantee.
- Components are functional and accessible by default.
- Hooks do not hide persistent side effects.
- Feature modules own their screens/types/tests.
- Use design tokens; no isolated magic color/spacing values.
- User-visible strings come from localization resources.
- Errors use typed domain envelopes and safe message keys.
- React state does not duplicate Room truth beyond explicit query/view cache.

6 Kotlin standards

- Coroutines with structured concurrency; no `GlobalScope`.
- IO on `Dispatchers.IO`; no blocking main-thread database/file work.
- Receivers call `goAsync()` and finish within bounded work.
- Every wake lock/service has owner, timeout and terminal cleanup.
- Sealed classes/value objects represent states and errors.
- Room writes use explicit transactions.

- Use `java.time` APIs with desugaring as needed; no ad hoc Calendar recurrence logic.
- PendingIntents explicit and immutable.
- Android components exported false unless documented.
- Domain calculators are pure and JVM-testable.

Formatting is enforced by a pinned Kotlin formatter and static analysis; rule suppressions need comments and review.

7 Naming

- UUID entity variables: `reminderId`, `assetId`, `occurrenceId`, `sessionId`.
- Absolute timestamps: `scheduledAt`, `createdAt`; local values: `localTime`, `zoneId`.
- Booleans describe state: `isInteractive`, `enabledIntent`, `canScheduleExact`.
- Operations use verbs: `reconcileScheduler`, `claimOccurrence`, `commitImport`.
- Avoid generic `Manager` unless component coordinates multiple services; prefer Repository, Calculator, Coordinator or Gateway.

8 Database and backup changes

Any persistence change requires in the same pull request:

1. Room migration and migration test;
2. updated schema documentation/ERD if structural;
3. backup DTO effect and reader/writer compatibility decision;
4. round-trip fixture update;
5. rollback/recovery assessment;
6. MR-21 traceability update;
7. ADR if semantics or portability change.

Destructive migration flags are prohibited in release.

9 Permission and background changes

A new permission, receiver, service, foreground-service type, job, alarm or exported component requires:

- user problem and least-privilege rationale;
- Android version/policy research from official sources;
- battery impact;
- privacy notice impact;
- denial/fallback UX;
- manifest/security tests;
- ADR when it changes the approved permission set.

No dependency may introduce a permission unnoticed; merged manifest is inspected in CI.

10 Testing expectations

Change	Minimum evidence
Pure recurrence/domain	Unit/property tests
Room entity/query	DAO + migration/query-plan test
Alarm receiver/action	Instrumentation with RN disabled and duplicate intents
UI component	Component semantics + visual/manual state
Permission/notification	API matrix scenario and denial fallback
Import/file	crash/failure injection and cleanup test
Backup	round trip plus malicious/conflict fixtures
Accessibility	TalkBack/scale impact note and test
Performance-sensitive	before/after benchmark against MR-15

Bug fixes add a regression test where feasible.

11 Documentation standards

The Markdown pack is source; PDFs are generated artifacts. Requirements use stable IDs. No TODO, TBD or unresolved placeholder is accepted in an Approved document. Diagrams use editable source plus rendered PNG/SVG. Current platform facts are linked in MR-22 with access date and refresh trigger.

Code comments explain why/invariants, not restate syntax. Public APIs and unusual lifecycle choices receive KDoc/TSDoc.

12 Accessibility review

Every component PR confirms name/role/state, focus order, 48 dp target, font scale and non-color state. Alarm/notification changes require manual TalkBack evidence. String additions include translator context and pseudo-locale review for critical flows.

13 Security and privacy review

Never commit personal media, real backup files, signing keys, keystores, private diagnostics or secrets. Fixtures are synthetic and licensed. Security-sensitive changes follow MR-12. Vulnerability reports are handled privately; do not open a public proof-of-concept issue that exposes users before a fix.

14 Dependencies

A dependency proposal states:

- exact capability needed;
- why platform/current dependencies are insufficient;
- license;
- maintenance/release status;
- transitive size and permissions;

- native/React compatibility;
- removal plan;
- security history where available.

Lockfile changes are reviewed. Avoid “utility” packages for trivial code. Update dependencies in focused PRs with alarm/backup regression suite.

15 UI implementation

Use the documented Material 3-inspired design system. Screens must implement loading, empty, error, offline/local and high-font states. Do not hardcode notification heads-up geometry. Do not use sacred/religious symbols as universal controls. Public screenshots use safe media.

16 Pull request template

A PR answers:

- Problem and linked requirements.
- User-visible behavior.
- Architecture/ADR impact.
- Permission, privacy, battery, backup and accessibility impact.
- Tests and devices run.
- Screenshots/recordings with synthetic data when UI changes.
- Migration/rollback.
- Known limitations.

17 Review checklist

Reviewer verifies requirement correctness, native/JS ownership, failure paths, concurrency/idempotency, cleanup, permission least privilege, safe logs, accessibility semantics, test quality, documentation and release impact. “Works on my phone” is not sufficient for alarm code.

18 Definition of done

A change is done when:

- implementation follows approved specs/ADR;
- tests pass and new risk has evidence;
- no inaccessible or unsafe failure path;
- schema/backup compatibility handled;
- no battery invariant regression;
- strings/localization and design states complete;
- docs/traceability/changelog updated;
- release manifest remains compliant;
- code reviewed with no unresolved blocking comment.

19 AI-assisted contributions

AI-generated code is reviewed under identical standards. The contributor is responsible for licensing, correctness, security and tests. Generated changes must cite the requirement/ADR they implement and cannot invent APIs, permissions or dependencies. MR-19 governs the development loop.

20 Community conduct

Use a respectful, inclusive code of conduct. The originating Islamic use case should be treated accurately and without mockery, while the general engine supports diverse lawful personal practices. Harassment, scraped personal data and copyrighted demo media are not accepted.

20.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.